

Improvement of lagrangian approach for multiphase flow

(Project for the course “CFD with OpenSource Software, 2014”)

Matteo Nobile

Department of Industrial Engineering
University of Rome Tor Vergata

- Introduction, objectives and methodology
- Theory and Implementation in OpenFOAM
- Test case and results

- Introduction, objectives and methodology
- Theory and Implementation in OpenFOAM
- Test case and results

Multiphase flow:

- dispersed multiphase flow

→ solid particles and liquid droplets in a carrier phase

- different approaches for modeling

→ Lagrangian - Eulerian

- libraries for lagrangian solution in OpenFOAM (version 2.2.2):

- ✓ two separate solutions: liquid spray (with breakup modeling) and multiphase spray (with surface reactions modeling)
- ✓ drag coefficient for non-deforming and non-evaporating droplets
- ✓ evaporation modeling based on superheated drop flash vaporization



Objectives:

1. Integration of multiphase capability in library for liquid spray modeling
2. New formulation for injection process
3. New formulation for evaporation process
4. New formulation for drag coefficient

Methodology:

1. Theoretical formulation
2. Implementation
3. Testing

- Introduction, objectives and methodology
- Theory and Implementation in OpenFOAM
- Test case and results

Multiphase spray modeling:

Implementation:

1. Multiphase variables (Ygas, YSolid, ...) added in the library for liquid spray modeling

2. Multiphase models added (surface reactions, ...)

```
template<class CloudType>
Foam::SprayCloud<CloudType>::SprayCloud
( SprayCloud<CloudType>& c, const word& name )
:
    .....
    atomizationModel_(c.atomizationModel_->clone()),
    breakupModel_(c.breakupModel_->clone()),
    stochasticCollisionModel_(c.stochasticCollisionModel_->clone()),
    mydevolatilisationModel_(c.mydevolatilisationModel_->clone()),
    mysurfaceReactionModel_(c.mysurfaceReactionModel_->clone()),
    dMassDevolatilisation_(c.dMassDevolatilisation_),
    dMassSurfaceReaction_(c.dMassSurfaceReaction_)
{}
```

```
template<class ParcelType>
Foam::SprayParcel<ParcelType>::
    SprayParcel(const SprayParcel<ParcelType>& p)
:
    ParcelType(p),
    d0_(p.d0_),
    position0_(p.position0_),
    liquidCore_(p.liquidCore_),
    - Reynolds_(p.Reynolds_),
    - Weber_(p.Weber_),
    - om2_(p.om2_),
    KHindex_(p.KHindex_),
    y_(p.y_),
    yDot_(p.yDot_),
    ms_(p.ms_),
    injector_(p.injector_),
    tMom_(p.tMom_),
    user_(p.user_),
    - Bt_(p.Bt_),
    - YGas_(p.YGas_),
    - YLiquid_(p.YLiquid_),
    - YSolid_(p.YSolid_),
    - canCombust_(p.canCombust_)
{}
```

Injection Process:

Critical Aspects:

1. Possibility of deactivating injection process
2. Match the total required mass to inject
3. Speed up the calculation

Injection Process:

Analysis:

1. Possibility of deactivating injection process

- Avoid floating point error in the method `averageParcelMass()`

```
Foam::scalar Foam::InjectionModelList<CloudType>::averageParcelMass()
{
    scalar mass = 0.0;
    scalar massTotal = 0.0;
    forAll(*this, i)
    {
        scalar mt = this->operator[](i).massTotal();
        mass += mt*this->operator[](i).averageParcelMass();
        massTotal += mt;
    }
    return mass/massTotal;
}
```

```
Foam::scalar Foam::InjectionModel<CloudType>::averageParcelMass()
{
    label nTotal = 0.0;
    if (this->owner().solution().transient())
    {
        nTotal = parcelsToInject(0.0, timeEnd() - timeStart());
    }
    else
    {
        nTotal = parcelsToInject(0.0, 1.0);
    }
    return massTotal_/nTotal;
}
```

when injection is deactivated:

```
Foam::label Foam::NoInjection<CloudType>::parcelsToInject
( const scalar, const scalar )
{ return 0; }
```



Injection Process:

Implementation:

1. Possibility of deactivating injection process

- Take advantage of the boolean active (false without injection)

```
Foam::scalar Foam::InjectionModelList<CloudType>::averageParcelMass()
{
    scalar mass = 0.0;
    scalar massTotal = 0.0;
    forAll(*this, i)
    {
        if (!this->operator[](i).active())
        {
            mass = 1.0;
            massTotal = 1.0;
        }
        else
        {
            scalar mt = this->operator[](i).massTotal();
            mass += mt*this->operator[](i).averageParcelMass();
            massTotal += mt;
        }
    }
    return mass/massTotal;
}
```

Injection Process:

Formulation:

2. Match the required total mass to inject

- More connection between input variables (injection profile $p(t)$, total mass m_{tot} , parcel flow rate $\dot{n}$, injection duration Δ)

injectable parcels
 at each time step k : $N_{avail} = \text{floor}((t_k - t_{k-1}) * \dot{n})$  $N_{avail} = \text{floor}\left(\frac{\int_k^k p(t)}{\int_0^\Delta p(t)} * N_{tot}\right)$

- Particles per parcel evaluated with the current available parcels

for $i = 1 : N_{avail}$
number of particles per parcel parcel i - th of N_{avail}

$$np_i = \frac{6V_{avail}}{N_{avail}\pi d_i^3} \quad \text{---} \quad np_i = \frac{6V_{avail}}{(N_{avail} - i + 1)\pi d_i^3}$$

Injection Process:

Implementation:

2. Match the required total mass to inject

```

--- a/$WM_PROJECT_USER_DIR/src/lagrangian/intermediate/.../InjectionModel.C
+++ b/$WM_PROJECT_DIR/src/lagrangian/intermediate/.../InjectionModel.C

    if ((time0 >= 0.0) && (time0 < duration_))
    {
-       scalar nTOT = floor(duration *parcelsPerSecond_);
-       scalar volFract = flowRateProfile_.integrate(time0, time1)/this->volumeTotal();
-       return floor(nTOT*volFract);
+       return floor((time1 - time0)*parcelsPerSecond_);
    }

+       scalar delayedVolume = 0;
+       .....
-       scalar density = cloud.constProps().rho0(); //Set density equal to rho0
-       scalar Vdisp = delayedVolume_ + newVolumeFraction*massTotal_/density;

template<class CloudType>
Foam::scalar Foam::InjectionModel<CloudType>::setNumberOfParticles
(
    .....
    case pbMass:
    {
-       scalar volumep = pi/6.0*pow3(diameter);
-       nP = floor(Vdispon/((parcels-iter)*volumep));
+       scalar volumeTot = massTotal_/rho;
+       nP = (volumeFraction*volumeTot + delayedVolume_)/(parcels*volumep);

        if (pPtr->nParticle() >= 1.0)
        {
            .....
-           Vdisp -= pPtr->nParticle()*pPtr->volume();
            .....
+           else
+           {
+               delayedVolume += pPtr->nParticle()*pPtr->volume();
+               delete pPtr;
        }
    }

```

Injection Process:

Analysis:

3. Speed up the calculation

- Avoid running the lagrangian solution when parcels has $n_p < 1$

```
if (!pPtr->move(td, dt))
{
    delete pPtr;
}
else
{
    if (pPtr->nParticle() >= 1.0)
    {
        td.cloud().addParticle(pPtr);
        massAdded += pPtr->nParticle()*pPtr->mass();
        parcelsAdded++;
    }
    else
    {
        delayedVolume += pPtr->nParticle()*pPtr->volume();
        delete pPtr;
    }
}
```



Injection Process:

Implementation:

3. Speed up the calculation

- Reverse the order of if statements

```
if (pPtr->nParticle() >= 1.0)
{
    if (!pPtr->move(td, dt))
    {
        delete pPtr;
    }
    else
    {
        td.cloud().addParticle(pPtr);
        massAdded += pPtr->nParticle()*pPtr->volume()*density;
        Vdisp -= pPtr->nParticle()*pPtr->volume(); //ADDED
        parcelsAdded++;
    }
}
else
{
    delete pPtr;
    deleted++;
}
```

Evaporation modeling:

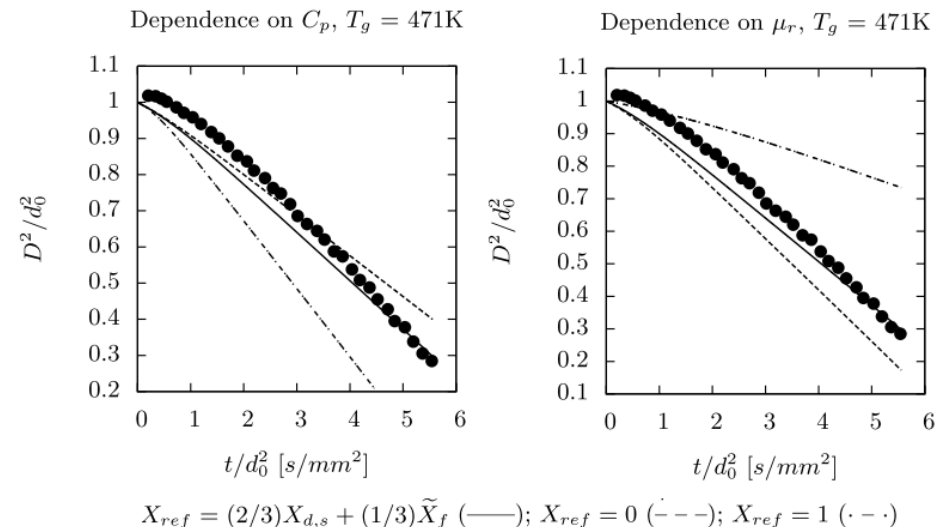
Critical Aspects:

1. Existing model based on superheated spray vaporization

2. Strong effect of thermophysical properties →

Main assumptions:

- No droplet internal circulation
- Infinitely high diffusivity (uniform temperature and composition)



Evaporation modeling:

Formulation: $X_{i,vap} = \frac{X_{i,liq} p_{sat,i}}{p}$ $Y_{i,vap} = \frac{PM_i X_{i,vap}}{PM_i X_{i,vap} + (1 - X_{i,vap}) PM_c}$

$$T_{ref} = \frac{2T + T_c}{3}$$

$$Y_{ref} = \frac{2Y_{i,vap} + Y_c}{3}$$

$$c_{p,ref} = Y_{ref} c_{p,i,vap} + (1 - Y_{ref}) c_{p,c}$$

$$rho_{ref} = \left(\frac{Y_{ref}}{\rho_{i,vap}} + \frac{(1 - Y_{ref})}{\rho_c} \right)^{-1}$$

$$\lambda_{ref} = \frac{X_{ref} \lambda_{i,vap}}{X_{ref} + (1 - X_{ref}) \frac{(1 + \sqrt{\frac{\lambda_{i,vap}}{\lambda_c} * (\frac{PM_c}{PM_j})^{0.25}})^2}{\sqrt{8(1 + \frac{PM_{i,vap}}{PM_c})}}} + \frac{(1 - X_{ref}) \lambda_c}{(1 + \sqrt{\frac{\lambda_c}{\lambda_{i,vap}} * (\frac{PM_c}{PM_{vap}})^{0.25}})^2 \sqrt{8(1 + \frac{PM_c}{PM_{vap}})}}$$

$$\mu_{ref} = \frac{X_{ref} \mu_{i,vap}}{X_{ref} + (1 - X_{ref}) \frac{(1 + \sqrt{\frac{\mu_{i,vap}}{\mu_c} * (\frac{PM_c}{PM_j})^{0.25}})^2}{\sqrt{8(1 + \frac{PM_{i,vap}}{PM_c})}}} + \frac{(1 - X_{ref}) \mu_c}{(1 + \sqrt{\frac{\mu_c}{\mu_{i,vap}} * (\frac{PM_c}{PM_{vap}})^{0.25}})^2 \sqrt{8(1 + \frac{PM_c}{PM_{vap}})}}$$

$$Re = \frac{\rho_c |u - u_c| d}{\mu_{ref}} \quad Sc = \frac{\mu_{ref}}{\rho_{ref} D_{i,vap}} \quad Gr = 9.81 d^3 \frac{\rho_c (T_c - T)}{T_c \mu_c} \quad Pr = \frac{\mu_{ref} c_{p,ref}}{\lambda_{ref}}$$

$$Nu_0 = 2.0009 + 0.514 \sqrt{\max(Re, \sqrt{Gr}) Pr^{\frac{1}{3}}} \quad Sh_0 = 2.0009 + 0.514 \sqrt{\max(Re, \sqrt{Gr}) Sc^{\frac{1}{3}}}$$

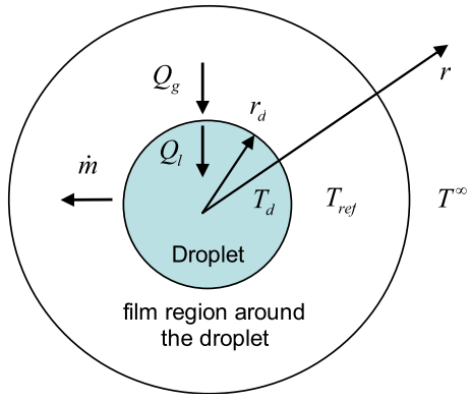
$$B_M = \frac{Y_{i,vap} - Y_c}{1 - Y_{i,vap}} \quad F_M = \frac{(1 + B_M)^{0.7}}{B_M} \log(1 + B_M) \quad Sh = 2 + \frac{Sh_0 - 2}{F_M}$$

$$\phi = \frac{c_{p,i,vap}}{c_{p,ref}} \frac{Sh}{Nu} \frac{Pr}{Sc} \quad B_T = (1 + B_M)^\phi - 1 \quad Nu = 2 + \frac{Sh_0 - 2}{F_T}$$



$$dM = \pi d Sh D_{i,vap} \rho_{ref} \log(1 + B_T) dt$$

$$dH = dM H_{vap}$$



Evaporation modeling:

Implementation:

1. Creation of new class NewLiquidEvaporation

(\$WM_PROJECT_USER_DIR/src/lagrangian/intermediate/submodels/Reacting/PhaseChangeModel/.)

2. Add Spalding heat transfer number

B_T in the constructor of template class

SprayParcel for speeding iterative

cycle each time step

```
template<class ParcelType>
Foam::SprayParcel<ParcelType>::
SprayParcel(const SprayParcel<ParcelType>& p)
:
ParcelType(p),
d0_(p.d0_),
position0_(p.position0_),
liquidCore_(p.liquidCore_),
- Reynolds_(p.Reynolds_),
- Weber_(p.Weber_),
- om2_(p.om2_),
KHindex_(p.KHindex_),
y_(p.y_),
yDot_(p.yDot_),
ms_(p.ms_),
injector_(p.injector_),
tMom_(p.tMom_),
user_(p.user_),
- Bt_(p.Bt_),
- YGas_(p.YGas_),
- YLiquid_(p.YLiquid_),
- YSolid_(p.YSolid_),
- canCombust_(p.canCombust_)
{}

```

Evaporation modeling:

Implementation:

3. Add new input variables to the virtual function calculate wherever it is reimplemented (the other phase change models) and called (type grep -rnw \$WM_PROJECT_DIR/src -e "phaseChange.calculate")

```

--- a/$WM_PROJECT_USER_DIR/src/.../NewLiquidEvaporation.C
+++ b/$WM_PROJECT_DIR/src/.../LiquidEvaporation.C

template<class CloudType>
- void Foam::NewLiquidEvaporation<CloudType>::calculate
+ void Foam::LiquidEvaporation<CloudType>::calculate
(
    const scalar dt,
    const label cellI,
-   scalar& Re,
+   const scalar Re,
    const scalar Pr,
    const scalar d,
    const scalar nu,
    const scalar T,
    const scalar Ts,
    const scalar pc,
    const scalar Tc,
        const scalarField& Yl,
    scalarField& dMassPC,
-   scalar& bt,
-   const scalar mu
) const

```

Drag coefficient calculation:

Critical Aspects:

1. Separate the calculation for different phases
2. Include effects of deformation and evaporation

Formulation (even alternatives, based on literature):

- More phases:
$$C_d = \begin{cases} \frac{24}{Re} (1 + \frac{1}{6} Re^{\frac{2}{3}}) & Re < 1000 \\ \frac{24}{Re} (1 + 0.15 Re^{0.687}) & Re < 1000 \\ 0.44 & Re > 1000 \end{cases}$$
- Only liquid phase:
$$C_d = \begin{cases} \frac{24}{Re} (1 + 0.13 Re^{0.82 - \log Re}) & Re < 20 \\ \frac{24}{Re} (1 + B_T)^{-0.2} (1 + 0.2 Re^{0.63}) (1 + 0.06 Re^{-0.12} We^{1.4}) & 20 < Re < 300 \\ \frac{24}{Re} (1 + 0.6 Re^{\frac{2}{3}}) (1 + 2.632 y) & Re > 300 \end{cases}$$

$$Re = \frac{\rho_{fl} |u - u_{fl}| d}{\mu_{film}}$$

Drag coefficient calculation:

Implementation:

1. Creation of new class DeformedSphereDrag

(\$WM_PROJECT_USER_DIR/src/lagrangian/intermediate/submodels/Kinematic/ParticleForces/Drag/.)

2. Add Reynolds Re and Weber We in

the constructor of template class

SprayParcel for analyzing parcel
motion in post processing

```
template<class ParcelType>
Foam::SprayParcel<ParcelType>::
SprayParcel(const SprayParcel<ParcelType>& p)
:
ParcelType(p),
d0_(p.d0_),
position0_(p.position0_),
liquidCore_(p.liquidCore_),
- Reynolds_(p.Reynolds_),
- Weber_(p.Weber_),
- om2_(p.om2_),
KHindex_(p.KHindex_),
y_(p.y_),
yDot_(p.yDot_),
ms_(p.ms_),
injector_(p.injector_),
tMom_(p.tMom_),
user_(p.user_),
- Bt_(p.Bt_),
- YGas_(p.YGas_),
- YLiquid_(p.YLiquid_),
- YSolid_(p.YSolid_),
- canCombust_(p.canCombust_)
{}

```

Drag coefficient calculation:

Implementation:

3. Add new input variables to the virtual function calcCoupled

wherever it is

reimplemented and

called (type grep -rnw

\$WM_PROJECT_DIR/src

-e "calcCoupled")

```
--- a/$WM_PROJECT_DIR/src/lagrangian/.../SphereDragForce.C
+++ b/$WM_PROJECT_USER_DIR/src/lagrangian/.../DeformedSphereDrag.C

template<class CloudType>
-Foam::forceSuSp Foam::SphereDragForce<CloudType>::calcCoupled
+Foam::forceSuSp Foam::DeformedSphereDrag<CloudType>::calcCoupled
(
    const typename CloudType::parcelType& p,
    const scalar dt,
    const scalar mass,
    const scalar Re,
    const scalar muc,
+   const scalar y,
+   const scalar YPhase,
+   const scalar Bt,
+   const scalar We
) const
{
```

Drag coefficient calculation:

Implementation:

4. Solve droplet oscillation equation (bool flag solveOscillationEq from false to true in constructor of BreakupModel)

```
--- a/$WM_PROJECT_USER_DIR/src/lagrangian/spray/.../BreakupModel.H
+++ b/$WM_PROJECT_DIR/src/lagrangian/spray/.../BreakupModel.H

//- Construct from dictionary
BreakupModel
(
    const dictionary& dict,
    CloudType& owner,
    const word& type,
    -    bool solveOscillationEq = true
    +    bool solveOscillationEq = false
);
```

New lagrangian sub-libraries

Steps:

1. create a directory lagrangian in the user src: mkdir
\$WM_PROJECT_USER_DIR/src/lagrangian
2. copy the original sub-libraries intermediate and spray in this directory:
cp -r \$WM_PROJECT_DIR/src/lagrangian/intermediate
\$WM_PROJECT_USER_DIR/src/lagrangian cp -r
\$WM_PROJECT_DIR/src/lagrangian/spray
\$WM_PROJECT_USER_DIR/src/lagrangian
3. copy the Allwmake files (or create them) from original lagrangian library; delete the lines not related to intermediate and spray: cp
\$WM_PROJECT_DIR/src/lagrangian/Allwmake
\$WM_PROJECT_USER_DIR/src/lagrangian

New lagrangian sub-libraries

Steps:

4. copy Allwmake, rename as Allwclean and replace wmake with wclean
cd \$WM_PROJECT_USER_DIR/src/lagrangian
cp Allwmake Allwclean
sed -i s/wmake/wclean/g Allwclean
5. clean the new lagrangian library:
./Allwclean
6. modify the destination and the name of the .so files of the new sub-libraries intermediate and spray, so that their \$WM_PROJECT_USER_DIR/src/lagrangian/*/Make/files (the * means intermediate and spray) is as follows:
LIB = \$(FOAM_USER_LIBBIN)/libbnlagrangian*

New lagrangian sub-libraries

Steps:

7. since the spray sub-library refers to the intermediate one, change in \$WM_PROJECT_USER_DIR/src/lagrangian/spray/Make/options file the location and the link of the original intermediate sub-library with the new one:

```
EXE_INC = \  
    ....  
    -I../intermediate/InInclude \  
    ....  
LIB_LIBS = \  
    ....  
    -L$(FOAM_USER_LIBBIN) \  
    -lbnlagrangianIntermediate \  
    ....
```

New lagrangian sub-libraries

Steps:

8. compile the new lagrangian library:

`./Allwmake`

9. insert and replace the new and the modified files in the sub-libraries. Then clean and compile again lagrangian library

Create solver with the new library

Steps:

1. copy an original solver that is similar to your applications (in this project it is sprayFoam) in the user's solvers directory. Rename the directory according to the name of the new solver (e.g. ureaFoam):

```
cp -r $WM_PROJECT_DIR/applications/solvers/lagrangian/sprayFoam  
$WM_PROJECT_USER_DIR/applications/solvers/ureaFoam
```

2. clean the new solver

```
cd $WM_PROJECT_USER_DIR/applications/solvers/ureaFoam  
wclean
```

Create solver with the new library

Steps:

3. rename the new solver:

```
mv  
$WM_PROJECT_USER_DIR/applications/solvers/ureaFoam/sprayFoam.  
C  
$WM_PROJECT_USER_DIR/applications/solvers/ureaFoam/ureaFoam.C  
sed -i s/sprayFoam/ureaFoam/g ureaFoam.C
```

4. delete the Allwmake and the sprayEngineFoam sub-solver:

```
rm  
$WM_PROJECT_USER_DIR/applications/solvers/ureaFoam/Allwmake  
rm -r  
$WM_PROJECT_USER_DIR/applications/solvers/ureaFoam/sprayEngine  
Foam
```

Create solver with the new library

Steps:

5. modify the destination and the name of the executable file of the new solver so that the new

\$WM_PROJECT_USER_DIR/applications/solvers/ureaFoam/Make/files
is as follows:

```
ureaFoam.C
```

```
EXE = $(FOAM_USER_APPBIN)/ureaFoam
```

6. copy the missing files for species and energy conservation

equations from the solver reactingParcelFoam: cp -r

```
$WM_PROJECT_DIR/applications/solvers/lagrangian/reactingParcelFoam/
YEqn.H
```

```
$WM_PROJECT_USER_DIR/applications/solvers/ureaFoam
```

Create solver with the new library

Steps:

cp -r

\$WM_PROJECT_DIR/applications/solvers/lagrangian/reactingParcelFoam
/EEqn.H \$WM_PROJECT_USER_DIR/applications/solvers/ureaFoam

7. in \$WM_PROJECT_USER_DIR/src/lagrangian/spray/Make/options file
delete the location of reactingParcelFoam and change the location
and the link of the original intermediate and spray sub-libraries with
the new one:

EXE_INC = \

....

-\$ (WM_PROJECT_USER_DIR)/src/lagrangian/intermediate/lnInclude \

-\$ (WM_PROJECT_USER_DIR)/src/lagrangian/spray/lnInclude \

....

Create solver with the new library

Steps:

```
EXE_LIBS = \
```

```
....
```

```
-L$(FOAM_USER_LIBBIN)\
```

```
-libnlagrangianIntermediate \
```

```
-libnlagrangianSpray \
```

```
....
```

8. compile the new solver:

```
wmake
```

9. check if the new solver is linked to the new sub-libraries:

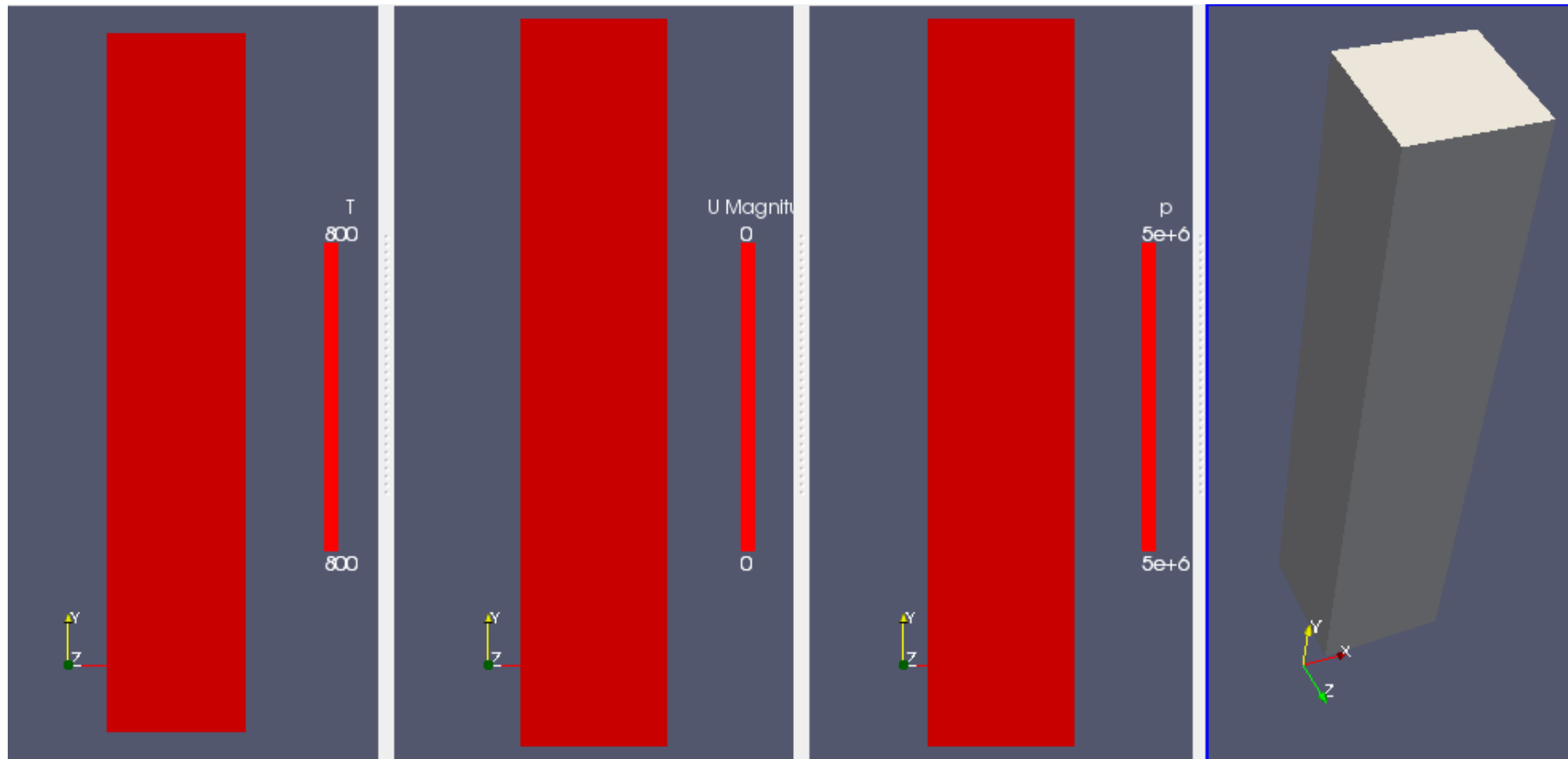
```
ldd 'which ureaFoam'
```

The correct output should be: ..., libnlagrangian.so,
libbnlagrangianIntermediate.so, libbnlagrangianSpray.so, ...

- Introduction, objectives and methodology
- Theory and Implementation in OpenFOAM
- Test case and results

Description of the reference case

New Aachen Bomb tutorial (non reacting + scaled injection):



Description of the tested cases

Test matrix:

CASE	INJECTION MODEL	EVAPORATION MODEL	DRAG COEFFICIENT MODEL
1	None	None	Sphere
2	Mass Total = 6e-8 kg	None	Sphere
3	Mass Total = 6e-7 kg	Original New	Sphere
4	Mass Total = 6e-7 kg	New	Deformed Sphere

Steps for run&post processing:

- blockMesh
- decomposePar (you need decomposeParDict)
- foamJob -parallel "solver"
- foamLog log (you should modify foamLog.db)
- gnuplot postPro.gplt

Case 1: Injection Deactivation

ORIGINAL FORMULATION

Floating Point Exception

```

Selecting AtomizationModel none
Selecting BreakupModel none
Selecting StochasticCollisionModel none
[0] #0 Foam::error::printStack(Foam::Ostream&)
[2] #0 Foam::error::printStack(Foam::Ostream&)
[3] #0 Foam::error::printStack(Foam::Ostream&)
in "/home/matteo/OpenFOAM/OpenFOAM-2.2.2/
platforms/linux64GccDP0pt/lib/libOpenFOAM.so"
[0] #1 Foam::sigFpe::sigHandler(int) in "/home/
matteo/OpenFOAM/OpenFOAM-2.2.2/platforms/
linux64GccDP0pt/lib/libOpenFOAM.so"
[0] #2 [1] #0 Foam::error::printStack
(Foam::Ostream&) in "/home/matteo/OpenFOAM/
OpenFOAM-2.2.2/platforms/linux64GccDP0pt/lib/
libOpenFOAM.so"
[2] #1 Foam::sigFpe::sigHandler(int) in "/lib/
x86_64-linux-gnu/libc.so.6"
[0] #3 in "/home/matteo/OpenFOAM/
OpenFOAM-2.2.2/platforms/linux64GccDP0pt/lib/
libOpenFOAM.so"

```

NEW FORMULATION

IF statement

```

....
Selecting BreakupModel none
Selecting StochasticCollisionModel none
Selecting devolatilisation model none
Selecting surface reaction model none
Average parcel mass: 1
Selecting radiationModel none
....

```

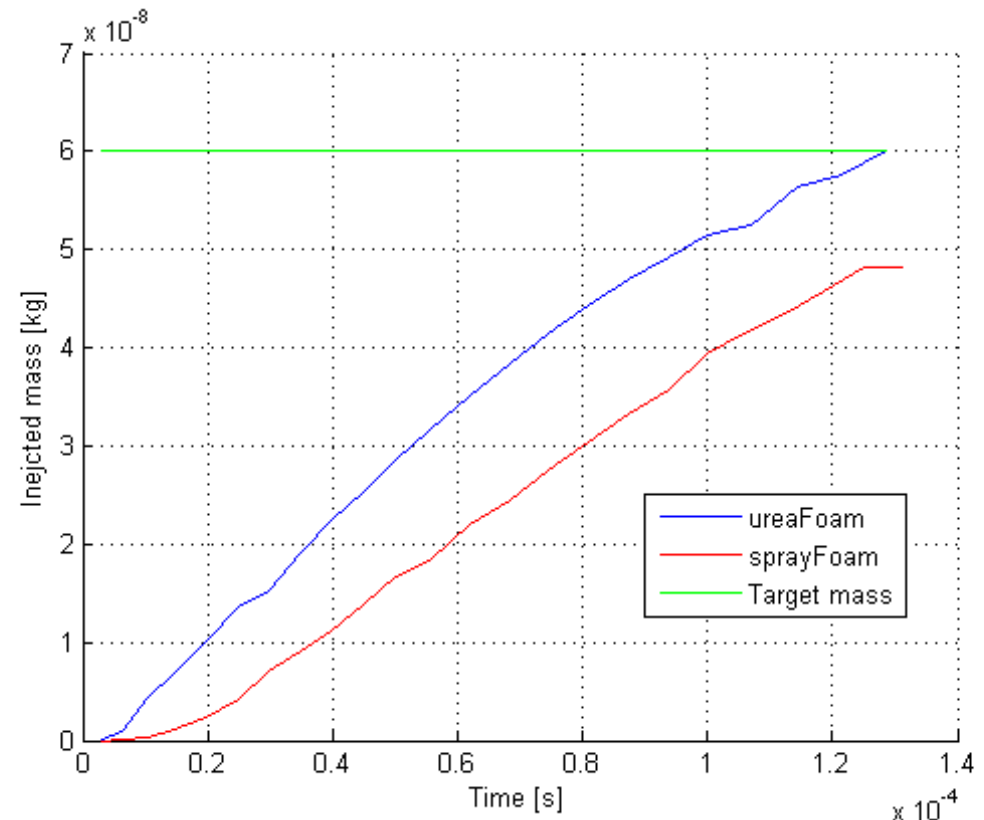
Case 2: Match total required mass to inject

- When the injectable mass per time step decreases (due to reduced time step and/or total required mass) the original formulation causes too low number of particles per parcel (<1)

$$np_i = \frac{6V_{avail}}{N_{avail}\pi d_i^3}$$



$$np_i = \frac{6V_{avail}}{(N_{avail} - i + 1)\pi d_i^3}$$

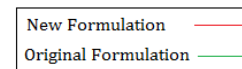
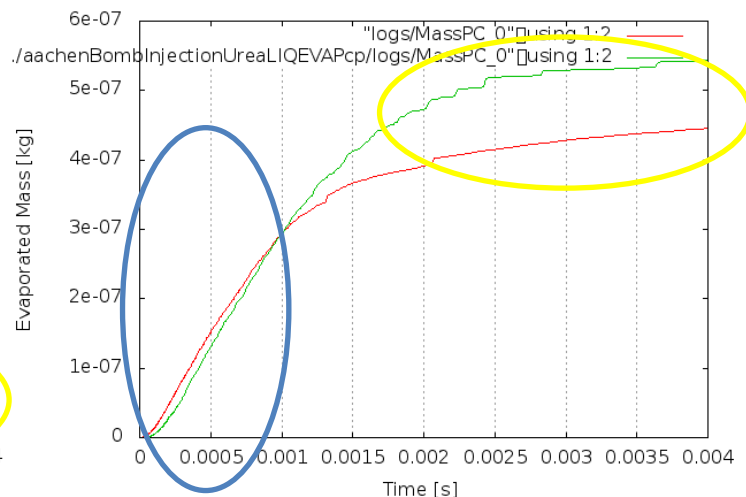
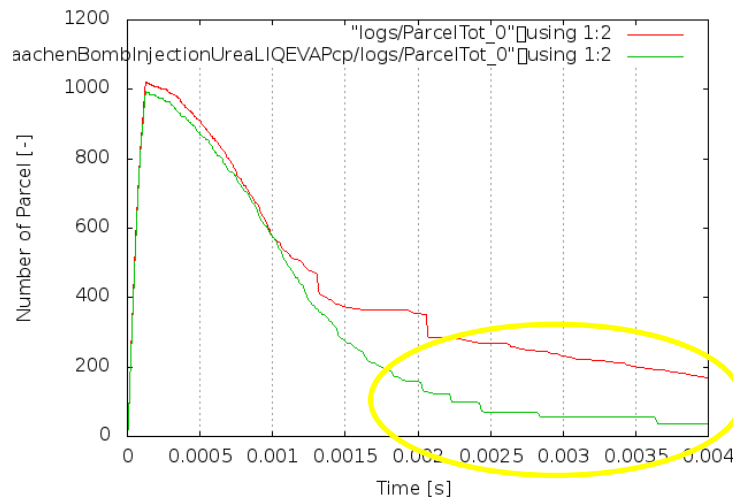
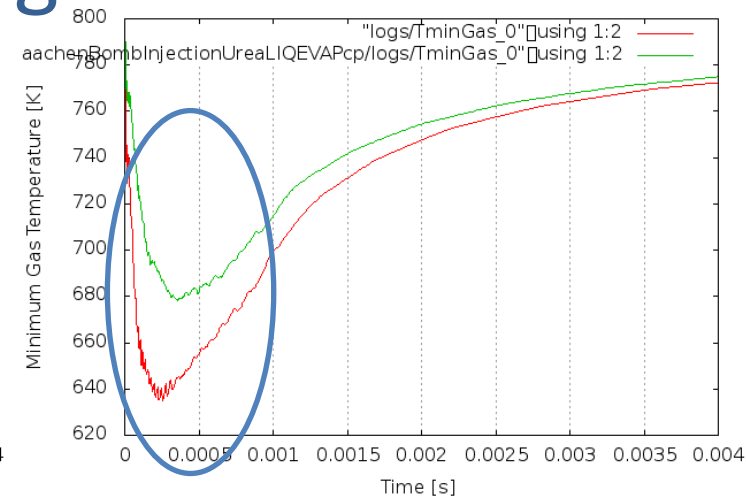
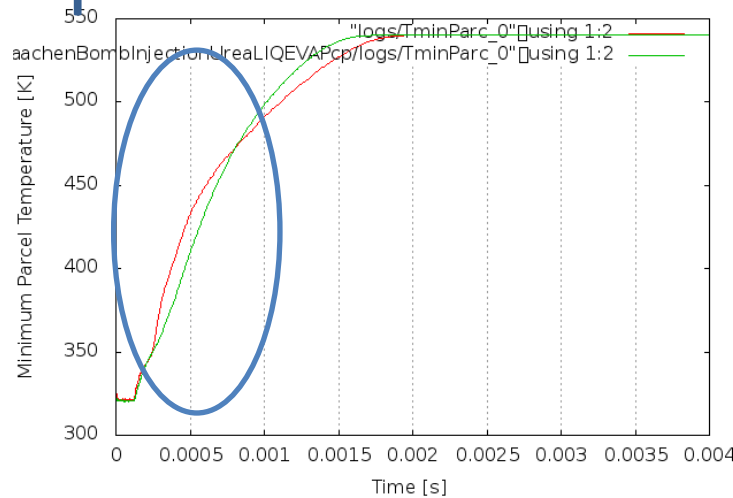


Case 3: Evaporation Modeling

Two
Phases:

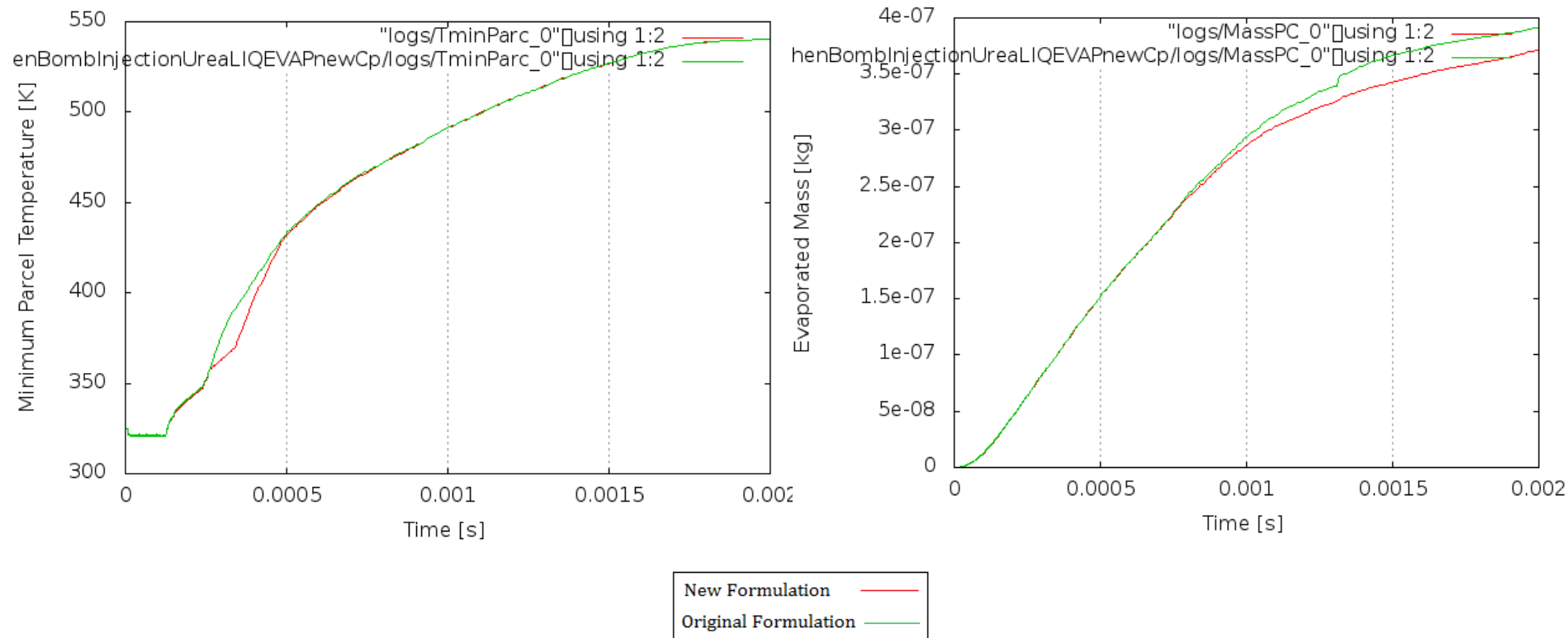
- Initial higher Vaporization with new formulation

- Lower rate after 1ms



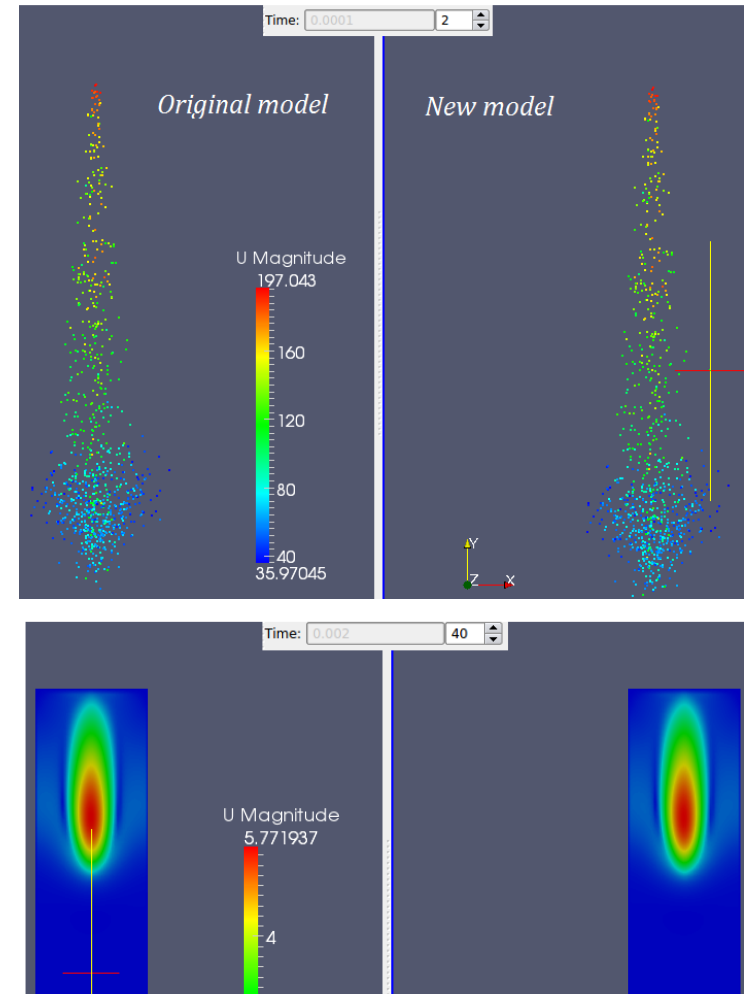
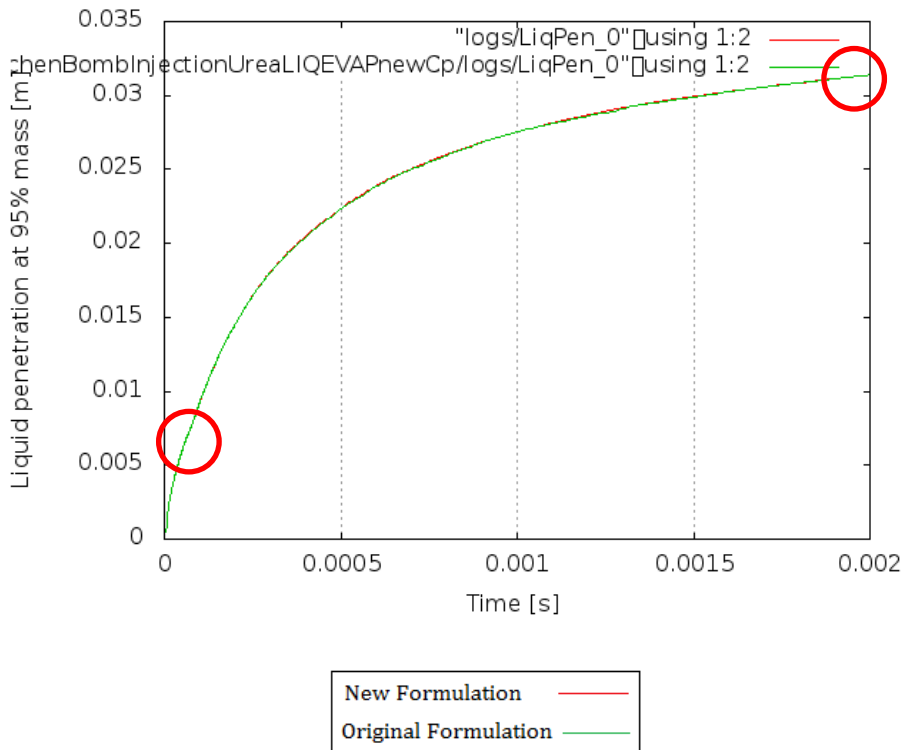
Case 4: Drag Coefficient Calculation

- Slightly lower evaporation rate with new drag formulation (no effect on temperature)



Case 4: Drag Coefficient Calculation

- Velocity distribution almost similar



Thank you for your attention!

Questions?